Contención de CPU y memoria en bases de datos documentales: MongoDB frente a PostgreSQL JSONB con límites de Docker (cgroups v2)

Reporte de métricas

Ángel Eduardo Chamé Pérez
Bases de Datos en la Nube
Universidad Politécnica de Chiapas (IT2iD)
Grupo 7-B
Profesor: Horacio Irán Solís Cisneros

1 de noviembre de 2026

Resumen

Se midió el comportamiento de MongoDB 9.0.4 y PostgreSQL 18.6 (JSONB) con 500 000 ventas sintéticas de un sistema de abarrotes, bajo tres perfiles de límites de Docker (4 CPU/4 GiB, 2 CPU/2 GiB y 1 CPU/1 GiB), dos consultas (Q1, agregación con desanidado, orientada a CPU; Q2, filtro con ordenamiento Top-1000, orientada a memoria) y 1, 10 y 50 clientes concurrentes. La matriz de 38 celdas se repitió en 3 rondas (114 corridas de 300 s, sin descartes): 87 corridas terminaron `ok`, 21 con volcado a disco (`spill`) y 6 con `oom`. Todas las corridas `spill` y `oom` son de PostgreSQL con Q1: su plan ordena los documentos completos antes de agrupar y escribe unos 0,8 GiB de archivos temporales por ejecución, su P99 es entre 2,0 y 13,6 veces el de MongoDB en las celdas que terminan sin OOM, y con 50 clientes agota la memoria en los perfiles B y C. En Q2 ambos motores escalan en proporción a la cuota de CPU; PostgreSQL es más rápido con 1 cliente y logra unas 1,3 veces más QPS con 50 clientes, y ninguna de las variantes de memoria de Q2 vuelca a disco. Por cada 1 000 documentos procesados (definición adoptada en este reporte; el enunciado no la precisa), Q1 con 1 cliente usa 8,7–11,8 ms de CPU en PostgreSQL frente a 2,9–3,2 ms en MongoDB, y Q2 entre 1,1 y 1,8 ms en ambos.

Índice

1. Especificaciones del host y del entorno Docker **3**

2. Método de medición y verificación del baseline **4**
2.1. Datos y consultas 4
2.2. Matriz y protocolo 4
2.3. Métricas 5
2.4. Verificación del baseline 5
2.5. Ejecución 5

3. Resultados consolidados **6**
Huella por cada 1 000 documentos procesados (M3) 8

4. Análisis gráfico **10**

5. Planes de ejecución **11**

6. Interpretación: límites de memoria, volcado a disco y throttling **13**
6.1. Respuestas a las preguntas del enunciado (M1, M2, M3) 13
6.2. Análisis 13

7. Recomendaciones técnicas **14**

Desviaciones y limitaciones **15**

A. Reproducción y archivos **16**

1. Especificaciones del host y del entorno Docker

Las mediciones se hicieron con Docker Desktop sobre WSL2 en Windows, no en Linux nativo (ver "Desviaciones y limitaciones"). La captura completa del entorno está en `evidence/entorno/` (`entorno.json` y salidas crudas).

Tabla 1: Host y entorno de ejecución

Equipo	HP Victus 15-fb2xxx (laptop), conectada a la corriente durante toda la matriz
CPU	AMD Ryzen 5 8645HS, 6 núcleos / 12 hilos, hasta 4,3 GHz
RAM del host	15,3 GiB
Disco	SSD NVMe WD PC SN560, 1 TB
Sistema	Windows 11 Home Single Language 10.0.26200 (64 bits); modo de energía "máximo rendimiento" con corriente alterna
WSL	2.6.3.0; kernel `6.6.87.2-microsoft-standard-WSL2`; sin `.wslconfig` (valores por defecto)
Máquina virtual de Docker	12 CPU, 7,39 GiB de RAM, 2 GiB de swap de la VM
Docker	Docker Desktop 4.97.0, Engine 29.9.2, Compose 5.6.1; cgroups v2 (driver `cgroupfs`); almacenamiento `overlayfs`
Cliente de carga	Python 3.12.10 en Windows (fuera de la VM); pymongo 4.19.0, psycopg 3.3.6

Tabla 2: Imágenes usadas (etiqueta exacta y digest) y versión que reporta cada motor

Imagen	Versión que reporta	Digest
`mongo:9.0.4`	9.0.4	(1)
`postgres:18.6`	18.6 (Debian 18.6-1.pgdg13+2)	(2)
`prom/prometheus:v3.15.1`	—	(3)
`prom/node-exporter:v1.12.1`	—	(4)
`ghcr.io/google/cadvisor:v0.60.6`	—	(5)

(1) `sha256:452bdaf2127b8ff851e31410a267809b9829eded219979f2210e7c7efb85a058`
(2) `sha256:3a4264520e121176619f54ff867d12774bc81e95e7b84db58d63bccdb6144b81`
(3) `sha256:fa3302279710be2f21a8301ec17a8b4f2301fd829bd113839b3b6b02b6f191a2`
(4) `sha256:1b4e4438faca4dd7e001dd445d161a4a2091b0fededa84093b3a8dfeae1f1be0`
(5) `sha256:b8e7d1093144fd088f425ff003d75a4aa405de075db78dae3bc563730b1bd07a`

Perfiles de recursos. Cada motor corre solo (el otro contenedor está apagado) y se recrea en cada corrida con los límites del perfil. Los límites se verificaron leyendo el cgroup real del contenedor (`evidence/verificacion/limites.json`) y se vuelven a leer y guardar en cada corrida (`meta.json`, campo `identidad.limites`).

Tabla 3: Perfiles y límites reales leídos del cgroup

Perfil	`cpus`	`mem_limit`	`cpu.max`	`memory.max`	`memory.swap.max`
A (`PROFILE_A_BASELINE`)	4	4096 MiB	`400000 100000`	4 294 967 296	0
B (`PROFILE_B_MEDIUM`)	2	2048 MiB	`200000 100000`	2 147 483 648	0
C (`PROFILE_C_SEVERE`)	1	1024 MiB	`100000 100000`	1 073 741 824	0

Configuración de los motores: la de cada imagen, sin ajustes de memoria ni de paralelismo. MongoDB se publica en el puerto 27018 del host (el 27017 lo ocupa otro servicio, detenido durante toda la matriz). PostgreSQL usa la base `benchmark` con collation C y `shm_size: 1g` (desviación declarada). La telemetría la recogen cAdvisor y Node Exporter, consultados por Prometheus cada segundo.

2. Método de medición y verificación del baseline

2.1. Datos y consultas

El documento es una venta del sistema de abarrotes del autor (datos sintéticos): folio, fecha ISO-8601 UTC, tipo de pago (`EFECTIVO`, `TRANSFERENCIA`, `FIADO`), total, bandera `es_fiado` y una lista `items` de 1 a 10 artículos (promedio 3,0); cada artículo lleva `producto.proveedor.{id, empresa}`, lo que da 4 niveles de anidamiento. Se generaron 500 000 ventas con semilla fija (20261101) en un solo archivo NDJSON (SHA-256 `59c35b36...2ad6a8c1`, 940,2 B por venta) que se cargó en ambos motores: colección `ventas` en MongoDB y tabla `ventas(id UUID PRIMARY KEY, data JSONB)` en PostgreSQL. Índices: en MongoDB, `fecha` y `folio`; en PostgreSQL, la llave primaria y un B-Tree sobre `(data->'fecha')`.

Listing 1: Q1 (CPU): desanidar, filtrar, calcular y agrupar

```
-- PostgreSQL
SELECT data->>'tipo_pago', item->'producto'->'proveedor'->>'id',
      SUM((item->>'cantidad')::numeric * (item->>'precio_unitario')::numeric),
      AVG((item->>'cantidad')::numeric * (item->>'precio_unitario')::numeric)
FROM ventas, jsonb_array_elements(data->'items') AS item
WHERE (item->>'precio_unitario')::numeric > 50
GROUP BY 1, 2
// MongoDB
[{$unwind: "$items"}, {$match: {"items.precio_unitario": {$gt: 50}}},
 {$project: {tipo_pago: 1, proveedor_id: "$items.producto.proveedor.id",
             subtotal: {$multiply: ["$items.cantidad", "$items.precio_unitario"]}}},
 {$group: {_id: {tipo_pago: "$tipo_pago", proveedor_id: "$proveedor_id"},
          suma_subtotal: {$sum: "$subtotal"}, promedio_subtotal: {$avg: "$subtotal"}}}]
```

Listing 2: Q2 (memoria): filtrar y ordenar sin índice, Top-1000

```
-- PostgreSQL (Q2: SET work_mem = '4MB'; Q2_RELAXED: SET work_mem = '64MB')
SELECT data FROM ventas
WHERE (data->>'es_fiado')::boolean = false AND data->>'fecha' >= '2026-01-01T00:00:00Z'
ORDER BY (data->>'total')::numeric DESC, (data->>'folio')::int ASC
LIMIT 1000
// MongoDB (Q2: allowDiskUse false; Q2_RELAXED: allowDiskUse true)
[{$match: {es_fiado: false, fecha: {$gte: "2026-01-01T00:00:00Z"}}},
 {$sort: {total: -1, folio: 1}}, {$limit: 1000}]
```

2.2. Matriz y protocolo

La matriz tiene 38 celdas: 2 motores × 3 perfiles × 2 consultas (Q1, Q2) × 3 concurrencias (1, 10, 50), más `Q2_RELAXED` con 50 clientes en el perfil C para cada motor. Se ejecutó en 3 rondas completas; dentro de cada ronda el orden de las celdas es aleatorio con semilla fija (`random.Random("20261101:ronda:<n>")`), para que la hora del día no se confunda con el perfil. El número de ronda es el `run_number` del CSV. Cada corrida sigue estos pasos (`runner/run_cell.py`):

1. Precondiciones: laptop conectada, servicio MongoDB de Windows detenido y al menos 60 GB libres en disco.
2. Recrear el contenedor del motor con los límites del perfil (el otro motor queda apagado) y verificar su identidad: puerto, versión exacta, un dato propio del contenedor (hostname o IP) y los límites del cgroup. Si no coinciden, la corrida aborta.
3. Vaciar la caché de páginas de la VM (`drop_caches`).
4. Lanzar c procesos cliente (`multiprocessing`, modo `spawn`), cada uno con su propia conexión, en lazo cerrado y sin pausa entre consultas.
5. 60 s de calentamiento, 300 s de medición y 60 s de enfriamiento. Timeout por consulta: 300 s. Las consultas que siguen en curso al cerrar la ventana se cancelan y no se cuentan.

2.3. Métricas

- **Throughput (QPS):** consultas exitosas que empezaron y terminaron dentro de la ventana, entre 300 s.
- **Latencias P50/P95/P99:** percentil por rango más cercano sobre las latencias de esas consultas. Con menos de 100 consultas el P99 coincide con el máximo y así se rotula; entre 100 y 999, el P99 se considera orientativo.
- **Throttling de CPU:** diferencia de `throttled_usec` de `cpu.stat` del cgroup entre el inicio y el fin de la ventana. En cgroups v2 este contador acumula el tiempo de espera de todas las CPU en que el grupo tenía hilos listos, por lo que puede superar los 300 s de reloj. Se complementa con la fracción de periodos CFS con throttling (`nr_throttled/nr_periods`).
- **RSS máxima:** máximo de `container_memory_rss` de cAdvisor en la ventana; también se guarda `memory.peak` del cgroup, que incluye caché de página.
- **Volcado a disco:** bytes temporales en la ventana: `temp_bytes` de `pg_stat_database` en PostgreSQL; `spillToDiskBytes` y `spilledDataStorageSize` de `serverStatus` en MongoDB.
- **OOM:** `oom_kill` de `memory.events` y la bandera `OOMKilled` de Docker.
- **Estado:** prioridad `oom` > `error` > `spill` > `ok`. Una corrida se descarta solo por una causa externa fijada de antemano (corriente, suspensión, caída de Docker); un OOM o un volcado son resultados, no descartes.

Cada corrida deja `meta.json` (identidad, contadores del cgroup, consultas, temporales), `latencias.csv` (todas las consultas), `serie_1s.csv` (RSS y throttling cada segundo) y, si hubo `oom` o `error`, `docker_logs.txt`.

2.4. Verificación del baseline

Antes del piloto se comprobó que ambos motores tienen los mismos datos y devuelven lo mismo (`runner/verificar_f2.py`; detalle en `evidence/verificacion/`). Todas las condiciones se cumplieron:

- 500 000 ventas en cada motor; 1 998 documentos de muestra idénticos entre motores.
- Tamaño por venta: 940,2 B en JSON, 984 B en BSON y 1 174,6 B en JSONB.
- Q1: los mismos 90 grupos y 674 686 artículos; diferencia máxima de las sumas $9{,}3 \times 10^{-10}$ (tolerancia: 1 centavo).
- Q2 y `Q2_RELAXED`: las mismas 1 000 filas en el mismo orden en ambos motores.
- Con 1 cliente en el perfil A, Q1 tarda 2,06 s en MongoDB y 3,44 s en PostgreSQL; Q2, 0,28 s y 0,25 s (una sola ejecución, no es medición).
- Los límites reales de los 6 contenedores coinciden con su perfil y `drop_caches` funciona en la VM (caché de 850 MB a 375 MB).

2.5. Ejecución

Un piloto de 3 corridas (resultados en `results/piloto/`, fuera de la matriz) fijó la duración real: unos 7,4 min por corrida. La matriz se lanzó el 1 de noviembre de 2026 a las 01:15 (hora de la Ciudad de México) y terminó a las 16:09. Según `results/bitacora.log`, hubo una sola pausa, entre corridas, de 11:59 a 12:49; al reanudar, el orquestador volvió a revisar el entorno. Las 114 corridas tienen fila en el CSV, sin descartes ni reintentos. Al cerrar cada ronda se hizo un respaldo con manifiesto SHA-256, verificado con `sha256sum -c`.

3. Resultados consolidados

Las tablas 4 y 5 resumen cada celda con la mediana de sus 3 rondas y el rango del P99 entre rondas. Se usa la mediana y no la media porque con 3 valores no se deja arrastrar por una ronda atípica. Las 114 filas individuales están en `results/benchmark_results.csv`; el resumen por celda, en `report/generado/celdas.csv`.

Estados. Las tres rondas dan exactamente los mismos estados: por ronda, 29 corridas `ok`, 7 `spill` y 2 `oom`, sin `error`. Todas las corridas `spill` y `oom` son de PostgreSQL con Q1; MongoDB no tuvo volcado a disco ni OOM en ninguna celda.

Tabla 4: Rendimiento por celda (ventana de 300 s)

	Motor	Perfil	c	Estados	QPS	P50 (ms)	P95 (ms)	P99 (ms)	P99 entre rondas mín.–máx. (ms)
Q1	MongoDB	A	1	3×ok	0,77	1 279	1 359	1 411‡	1 356–1 435
Q1	MongoDB	A	10	3×ok	1,72	5 708	5 923	6 090‡	5 806–6 396
Q1	MongoDB	A	50	3×ok	1,48	30 704	33 419	35 077‡	34 592–35 198
Q1	MongoDB	B	1	3×ok	0,76	1 304	1 387	1 429‡	1 419–1 571
Q1	MongoDB	B	10	3×ok	0,87	10 995	11 505	11 808‡	11 594–12 393
Q1	MongoDB	B	50	3×ok	0,67	58 995	64 498	66 694‡	66 497–67 097
Q1	MongoDB	C	1	3×ok	0,63	1 461	2 027	2 063‡	1 988–2 148
Q1	MongoDB	C	10	3×ok	0,42	22 293	23 801	24 799‡	23 503–25 702
Q1	MongoDB	C	50	3×ok	0,30	115 995	127 894	137 694†	133 193–138 096
Q1	PostgreSQL	A	1	3×spill	0,37	2 676	2 744	2 790‡	2 758–2 941
Q1	PostgreSQL	A	10	3×spill	0,17	44 561	80 579	82 581†	82 456–86 205
Q1	PostgreSQL	A	50	3×spill	0,01	179 946	210 294	210 294†	188 248–211 003
Q1	PostgreSQL	B	1	3×spill	0,25	3 822	5 038	5 343†	3 283–5 789
Q1	PostgreSQL	B	10	3×spill	0,13	62 638	74 523	74 662†	70 775–77 263
Q1	PostgreSQL	B	50	3×oom	0,01	133 112	157 157	157 157†	131 864–176 270
Q1	PostgreSQL	C	1	3×spill	0,14	7 290	8 179	8 497†	8 233–11 142
Q1	PostgreSQL	C	10	3×spill	0,08	93 100	104 898	105 002†	97 429–110 414
Q1	PostgreSQL	C	50	3×oom	0,00	—	—	—	—
Q2	MongoDB	A	1	3×ok	4,09	243	257	270	269–278
Q2	MongoDB	A	10	3×ok	10,70	908	1 086	1 108	1 105–1 116
Q2	MongoDB	A	50	3×ok	9,74	5 011	7 517	8 587	8 516–8 599
Q2	MongoDB	B	1	3×ok	3,95	250	275	289	282–292
Q2	MongoDB	B	10	3×ok	5,55	1 794	2 091	2 193	2 108–2 415
Q2	MongoDB	B	50	3×ok	5,09	9 294	15 498	18 087	17 705–18 295
Q2	MongoDB	C	1	3×ok	3,59	248	502	587	520–593
Q2	MongoDB	C	10	3×ok	2,80	3 499	4 105	4 503‡	4 408–4 506
Q2	MongoDB	C	50	3×ok	2,59	17 804	27 504	31 686‡	31 498–35 196
Q2	PostgreSQL	A	1	3×ok	6,65	149	163	175	175–177
Q2	PostgreSQL	A	10	3×ok	10,22	896	1 410	1 524	1 520–1 579
Q2	PostgreSQL	A	50	3×ok	12,93	3 678	5 523	6 755	6 719–6 817
Q2	PostgreSQL	B	1	3×ok	5,54	183	202	218	207–219
Q2	PostgreSQL	B	10	3×ok	5,64	1 597	2 592	2 792	2 789–2 916
Q2	PostgreSQL	B	50	3×ok	6,68	6 978	11 117	12 907	12 805–12 999
Q2	PostgreSQL	C	1	3×ok	2,87	328	398	411‡	402–460
Q2	PostgreSQL	C	10	3×ok	2,88	3 084	5 097	5 703‡	5 307–5 714
Q2	PostgreSQL	C	50	3×ok	3,42	12 992	21 732	25 294‡	24 796–26 290
Q2R	MongoDB	C	50	3×ok	2,62	17 286	27 297	31 795‡	30 406–34 405
Q2R	PostgreSQL	C	50	3×ok	3,45	12 887	21 610	25 389	25 097–25 390

Q2R: `Q2_RELAXED`. Mediana de las 3 rondas por celda; P99 mín.–máx. entre rondas. Percentiles por rango más cercano sobre las consultas exitosas de la ventana (D-007): † menos de 100 consultas en alguna ronda, el P99 es igual al máximo; ‡ entre 100 y 999, P99 orientativo. "—": sin consultas terminadas.

Tabla 5: Recursos por celda (ventana de 300 s)

	Motor	Perfil	c	Throttling (s)	Periodos con throttling (%)	RSS máx. (MiB)	Pico cgroup (MiB)	Temporales (GiB)	Consultas ok
Q1	MongoDB	A	1	0	0	744	1 102	0,0	232
Q1	MongoDB	A	10	1 871	100	816	1 170	0,0	517
Q1	MongoDB	A	50	2 350	100	978	1 335	0,0	443
Q1	MongoDB	B	1	4	6	757	1 122	0,0	228
Q1	MongoDB	B	10	2 527	100	812	1 167	0,0	260
Q1	MongoDB	B	50	2 956	100	908	1 263	0,0	202
Q1	MongoDB	C	1	74	63	547	899	0,0	188
Q1	MongoDB	C	10	2 884	100	585	939	0,0	127
Q1	MongoDB	C	50	3 228	100	710	1 024	0,0	90
Q1	PostgreSQL	A	1	0	0	102	1 716	87,7	110
Q1	PostgreSQL	A	10	159	14	566	4 096	51,6	50
Q1	PostgreSQL	A	50	179	14	1 972	4 096	65,9	4
Q1	PostgreSQL	B	1	83	19	102	1 703	60,1	75
Q1	PostgreSQL	B	10	688	46	568	2 048	43,1	38
Q1	PostgreSQL	B	50	990	56	1 787	2 048	n/m	4
Q1	PostgreSQL	C	1	147	53	103	1 024	33,1	41
Q1	PostgreSQL	C	10	2 232	100	569	1 024	32,5	24
Q1	PostgreSQL	C	50	2 637	86	966	1 024	n/m	0
Q2	MongoDB	A	1	0	0	500	797	0,0	1 226
Q2	MongoDB	A	10	1 737	100	552	845	0,0	3 211
Q2	MongoDB	A	50	2 356	100	641	935	0,0	2 923
Q2	MongoDB	B	1	2	5	494	706	0,0	1 184
Q2	MongoDB	B	10	2 453	100	542	840	0,0	1 665
Q2	MongoDB	B	50	2 969	100	622	836	0,0	1 527
Q2	MongoDB	C	1	59	30	536	835	0,0	1 077
Q2	MongoDB	C	10	2 838	100	551	836	0,0	840
Q2	MongoDB	C	50	3 254	100	624	917	0,0	777
Q2	PostgreSQL	A	1	0	0	24	497	0,0	1 995
Q2	PostgreSQL	A	10	1 955	100	92	580	0,0	3 067
Q2	PostgreSQL	A	50	2 322	100	298	826	0,0	3 880
Q2	PostgreSQL	B	1	140	96	23	497	0,0	1 663
Q2	PostgreSQL	B	10	2 600	100	93	581	0,0	1 691
Q2	PostgreSQL	B	50	2 935	100	298	824	0,0	2 005
Q2	PostgreSQL	C	1	491	100	23	497	0,0	861
Q2	PostgreSQL	C	10	2 981	100	93	581	0,0	864
Q2	PostgreSQL	C	50	3 237	100	298	820	0,0	1 027
Q2R	MongoDB	C	50	3 256	100	623	920	0,0	785
Q2R	PostgreSQL	C	50	3 235	100	298	819	0,0	1 036

Q2R: `Q2_RELAXED`. Mediana de las 3 rondas salvo RSS y pico del cgroup (máximo). Throttling: delta de `throttled_usec` de `cpu.stat`, que suma el tiempo de todas las CPU del grupo y puede superar los 300 s de la ventana. Periodos con throttling: `nr_throttled`/`nr_periods`. RSS: cAdvisor (`container_memory_rss`); pico del cgroup: `memory.peak`, que incluye caché de página y memoria compartida desde el arranque del contenedor. Temporales: `pg_stat_database.temp_bytes` (PostgreSQL) o `spillToDiskBytes` (MongoDB); n/m: no medido (PostgreSQL reinicia sus estadísticas tras un OOM kill).

Q1 (CPU). MongoDB completa Q1 en todas las celdas. Con 1 cliente, su P99 va de 1 411 ms (perfil A) a 2 063 ms (C); con 50 clientes, de 35 077 ms (A) a 137 694 ms (C). PostgreSQL es más lento en todas las celdas de Q1 y vuelca a disco en todas las que terminan:

- Con 1 cliente: P99 de 2 790 ms (A), 5 343 ms (B) y 8 497 ms (C), entre 2,0 y 4,1 veces el de MongoDB.
- Con 10 clientes: 0,17, 0,13 y 0,08 QPS, frente a 1,72, 0,87 y 0,42 de MongoDB.

- Con 50 clientes: en el perfil A solo terminan 3 o 4 consultas por ventana (0,01 QPS; el P99 de 210 294 ms es el máximo). En B, las tres rondas terminan en `oom` (5 OOM kills en total) y apenas 4 consultas por ventana. En C, las tres rondas terminan en `oom` (144 OOM kills) sin ninguna consulta exitosa.
- Volumen temporal: entre 32,5 y 87,7 GiB por ventana de 300 s; 1 140 GiB en toda la matriz.

Q2 (memoria). Ningún motor vuelca a disco. Con 1 cliente PostgreSQL tiene menor P99 en los tres perfiles (175, 218 y 411 ms, frente a 270, 289 y 587 ms de MongoDB). Con 50 clientes PostgreSQL da más QPS (12,93, 6,68 y 3,42, frente a 9,74, 5,09 y 2,59) y menor P99 (6 755, 12 907 y 25 294 ms, frente a 8 587, 18 087 y 31 686 ms). Con 10 clientes los dos quedan cerca: MongoDB tiene menor P99 en los tres perfiles.

Variante relajada de Q2. En el perfil C con 50 clientes, `Q2_RELAXED` (`allowDiskUse: true` / `work_mem = 64MB`) no cambia el resultado: MongoDB 2,62 QPS y P99 de 31 795 ms (frente a 2,59 y 31 686 con Q2); PostgreSQL 3,45 QPS y 25 389 ms (frente a 3,42 y 25 294). La sección 5 muestra por qué.

Repetibilidad. El rango del P99 entre rondas es estrecho en casi todas las celdas; por ejemplo, MongoDB A Q2 con 1 cliente da 278, 269 y 270 ms. Las celdas con más dispersión son las de PostgreSQL Q1, donde el disco interviene (B con 1 cliente: de 3 283 a 5 789 ms).

Huella por cada 1 000 documentos procesados (M3)

La tabla 6 aplica la siguiente *definición adoptada en este reporte (el enunciado no la precisa)*:

- **Documento procesado:** una venta leída por la consulta. Por consulta se toman los documentos examinados según los planes de la sección 5 (perfil A, 1 cliente): Q1 lee 500 000 en ambos motores; Q2 y `Q2_RELAXED`, 227 094 en ambos (MongoDB: `totalDocsExamined`; PostgreSQL: filas del `Index Scan` más las eliminadas por el filtro). Se asume que ese número no cambia con el perfil ni con la concurrencia.
- **Documentos procesados en la ventana:** consultas ok de la ventana (campo `medidas_ok` de `meta.json`) por documentos por consulta.
- **CPU:** delta de `usage_usec` de `cpu.stat` del cgroup entre el inicio y el fin de la ventana, en ms por cada 1 000 documentos procesados.
- **Memoria:** la RSS máxima de la ventana (`max_rss_mb`) y la razón "MiB de RSS por cada 1 000 documentos por segundo" (RSS máxima entre miles de documentos procesados por segundo). Esta razón es una interpretación de la frase "huella de memoria por cada 1 000 documentos" del enunciado: la RSS es un nivel, no un consumo acumulable. La tabla muestra la mediana entre rondas de esa RSS máxima ("RSS mediana"); la tabla 5 muestra el máximo entre rondas.
- Por celda, la mediana de las 3 rondas de cada valor.

Tabla 6: Huella de CPU y memoria por cada 1 000 documentos procesados (M3)

	Motor	Perfil	c	Consultas ok (mediana)	Docs. por consulta	CPU en la ventana (s)	ms de CPU por 1 000 docs.	RSS mediana (MiB)	MiB de RSS por 1 000 docs./s
Q1	MongoDB	A	1	232	500 000	332	2,9	737	1,9
Q1	MongoDB	A	10	517	500 000	1 200	4,6	815	0,9
Q1	MongoDB	A	50	443	500 000	1 199	5,4	974	1,3
Q1	MongoDB	B	1	228	500 000	331	2,9	750	2,0
Q1	MongoDB	B	10	260	500 000	600	4,6	793	1,9
Q1	MongoDB	B	50	202	500 000	598	5,9	901	2,7

continúa

	Motor	Perfil	c	Consultas ok (mediana)	Docs. por consulta	CPU en la ventana (s)	ms de CPU por 1 000 docs.	RSS mediana (MiB)	MiB de RSS por 1 000 docs./s
Q1	MongoDB	C	1	188	500 000	300	3,2	544	1,7
Q1	MongoDB	C	10	127	500 000	299	4,7	584	2,8
Q1	MongoDB	C	50	90	500 000	298	6,6	699	4,7
Q1	PostgreSQL	A	1	110	500 000	480	8,7	102	0,6
Q1	PostgreSQL	A	10	50	500 000	450	17,0	564	6,8
Q1	PostgreSQL	A	50	4	500 000	486	278,3†	1 969	295,8†
Q1	PostgreSQL	B	1	75	500 000	352	9,4	102	0,8
Q1	PostgreSQL	B	10	38	500 000	408	23,3	566	8,9
Q1	PostgreSQL	B	50	4	500 000	499	249,6*†	1 787	268,1*†
Q1	PostgreSQL	C	1	41	500 000	241	11,8	101	1,5
Q1	PostgreSQL	C	10	24	500 000	300	25,0	566	14,1
Q1	PostgreSQL	C	50	0	500 000	277	n/c	966	n/c
Q2	MongoDB	A	1	1 226	227 094	305	1,1	494	0,5
Q2	MongoDB	A	10	3 211	227 094	1 200	1,6	551	0,2
Q2	MongoDB	A	50	2 923	227 094	1 200	1,8	639	0,3
Q2	MongoDB	B	1	1 184	227 094	305	1,1	493	0,5
Q2	MongoDB	B	10	1 665	227 094	600	1,6	540	0,4
Q2	MongoDB	B	50	1 527	227 094	597	1,7	619	0,5
Q2	MongoDB	C	1	1 077	227 094	281	1,1	530	0,6
Q2	MongoDB	C	10	840	227 094	300	1,6	547	0,9
Q2	MongoDB	C	50	777	227 094	296	1,7	618	1,0
Q2	PostgreSQL	A	1	1 995	227 094	726	1,6	23	0,0
Q2	PostgreSQL	A	10	3 067	227 094	1 200	1,7	92	0,0
Q2	PostgreSQL	A	50	3 880	227 094	1 200	1,4	297	0,1
Q2	PostgreSQL	B	1	1 663	227 094	599	1,6	20	0,0
Q2	PostgreSQL	B	10	1 691	227 094	600	1,6	92	0,1
Q2	PostgreSQL	B	50	2 005	227 094	600	1,3	298	0,2
Q2	PostgreSQL	C	1	861	227 094	300	1,5	23	0,0
Q2	PostgreSQL	C	10	864	227 094	299	1,5	93	0,1
Q2	PostgreSQL	C	50	1 027	227 094	299	1,3	298	0,4
Q2R	MongoDB	C	50	785	227 094	296	1,7	622	1,0
Q2R	PostgreSQL	C	50	1 036	227 094	300	1,3	298	0,4

Definición adoptada en este reporte (el enunciado no la precisa). Q2R: `Q2_RELAXED`. Documentos procesados = consultas ok de la ventana × documentos examinados por consulta según el plan (perfil A, 1 cliente; se asume constante entre perfiles y concurrencias). CPU: delta de `usage_usec` de `cpu.stat` en la ventana de 300 s (todas las CPU del contenedor; incluye las consultas en vuelo canceladas al cierre y no incluye el cliente). RSS mediana: mediana entre las 3 rondas del `max_rss_mb` del CSV (RSS máxima de cada ventana); la tabla de recursos muestra en cambio el máximo entre rondas. Mediana de las 3 rondas por celda. n/c: sin consultas ok, no se puede calcular. * celda con OOM: el CPU incluye los reinicios y la recuperación del motor. † menos de 10 consultas ok en alguna ronda: el CPU de las consultas en vuelo pesa mucho en el cociente.

Lo que muestran los números. En Q1 con 1 cliente, PostgreSQL gasta entre 3,0 y 3,7 veces el CPU por cada 1 000 documentos de MongoDB: 8,7, 9,4 y 11,8 ms en los perfiles A, B y C, frente a 2,9, 2,9 y 3,2 ms. Con más clientes el costo sube en ambos motores (MongoDB A: 2,9, 4,6 y 5,4 ms con 1, 10 y 50 clientes; PostgreSQL A: 8,7 y 17,0 ms con 1 y 10). Las celdas de PostgreSQL Q1 con 50 clientes (278,3 y 249,6 ms en A y B) se calculan con 3 o 4 consultas y con el CPU de las consultas canceladas al cierre, y en B además con reinicios por OOM: no son comparables con las demás. En C no hay consultas ok y la huella no se puede calcular. En Q2 los dos motores quedan cerca, entre 1,1 y 1,8 ms por cada 1 000 documentos: con 1 cliente MongoDB gasta menos (1,10 frente a 1,60 ms en A) y con 50 clientes PostgreSQL gasta menos (1,36 frente a 1,81 ms en A; 1,28 frente a 1,68 ms en C). `Q2_RELAXED` da lo mismo que Q2 (MongoDB 1,66 y PostgreSQL 1,28 ms en C con 50 clientes). El perfil cambia poco el CPU por documento con 1 cliente, salvo en PostgreSQL Q1, que sube de 8,7 a 11,8 ms de A a C. En memoria, la RSS mediana de MongoDB está entre 493 y 974 MiB en todas las celdas, así que su razón por documentos/s sube cuando el throughput baja (Q1 con 50 clientes: 1,3, 2,7 y 4,7 MiB en A, B y C). La RSS mediana de PostgreSQL en Q2 es

mucho menor (entre 20 y 298 MiB, con razones de 0,015 a 0,38 MiB). *Hipótesis no verificada:* parte de la diferencia de RSS se debe a que `container_memory_rss` no cuenta la memoria compartida, donde PostgreSQL guarda sus `shared_buffers`; por eso la comparación de memoria entre motores debe leerse con cautela.

4. Análisis gráfico

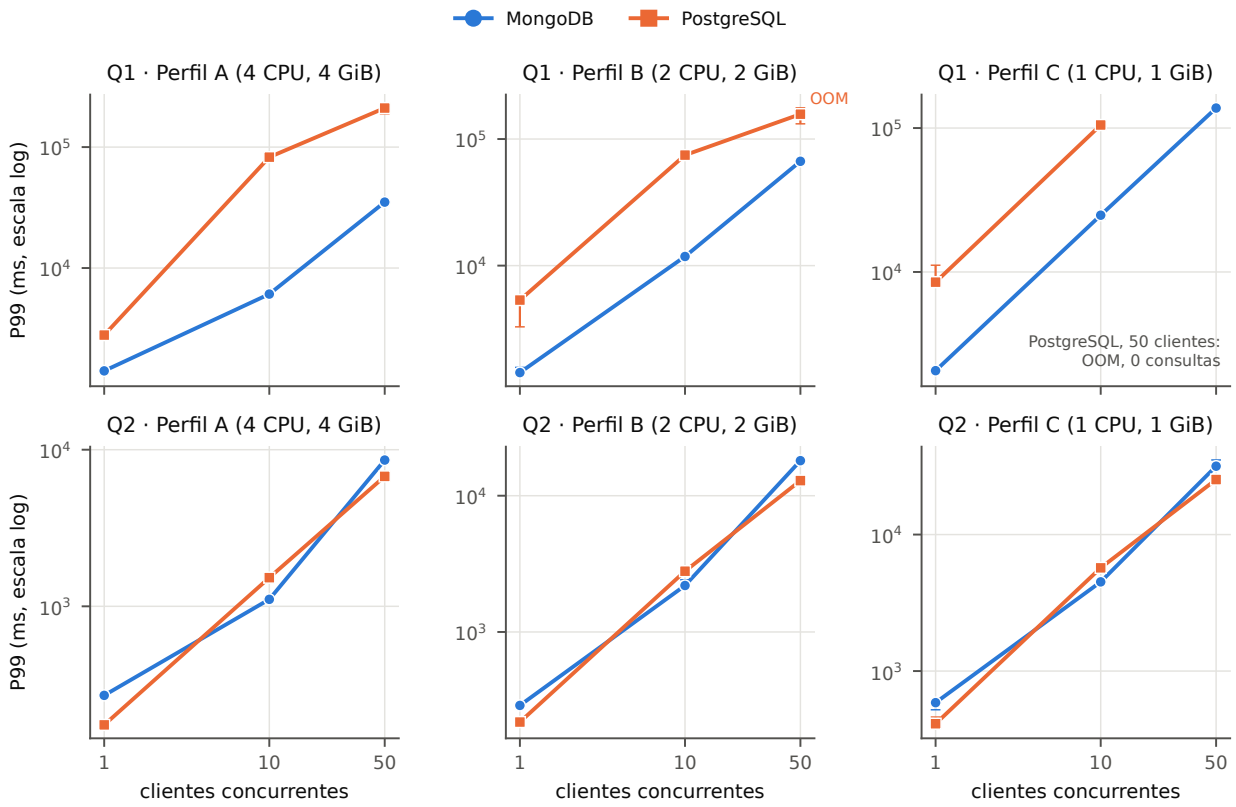


Figura 1: P99 frente a concurrencia (mediana de 3 rondas; las barras muestran el mínimo y el máximo entre rondas). Escalas logarítmicas. Todas las corridas de PostgreSQL Q1 volcaron a disco; con 50 clientes, B terminó en OOM y C en OOM sin consultas exitosas.

P99 frente a concurrencia (figura 1). En Q2 las curvas de los dos motores casi se superponen y crecen de forma aproximadamente lineal con la concurrencia: en lazo cerrado, una vez saturada la cuota de CPU, cada cliente adicional se forma en la cola y la latencia aumenta en proporción. En Q1 la curva de PostgreSQL queda siempre arriba de la de MongoDB, y la distancia es mayor con 10 clientes que con 1 (13,6 veces en el perfil A), porque las ejecuciones simultáneas compiten por el mismo disco.

Throttling frente a QPS (figura 2). Se forman dos grupos. Con 10 y 50 clientes, las corridas limitadas por CPU (todas las de MongoDB y las de Q2 en PostgreSQL) tienen throttling en el 100 % de los periodos y acumulan entre 1 700 y 3 300 s por ventana, más cuanto menor es la cuota. Su QPS es casi proporcional a la cuota: Q2 con 50 clientes da 12,93/6,68/3,42 QPS en PostgreSQL y 9,74/5,09/2,59 en MongoDB para 4/2/1 CPU (razón cercana a 3,8:2:1). En cambio, PostgreSQL Q1 tiene poco throttling y además muy bajo QPS: en el perfil A con 10 clientes, solo el 14 % de los periodos tienen throttling. Esa consulta espera al disco, no a la CPU. Con 1 cliente, el throttling es casi nulo en A y B, pero no en C: MongoDB Q1 tiene throttling en el 63 % de los periodos y su P99 sube de 1 411 a 2 063 ms aunque solo haya un cliente. Como el throttling solo ocurre cuando el

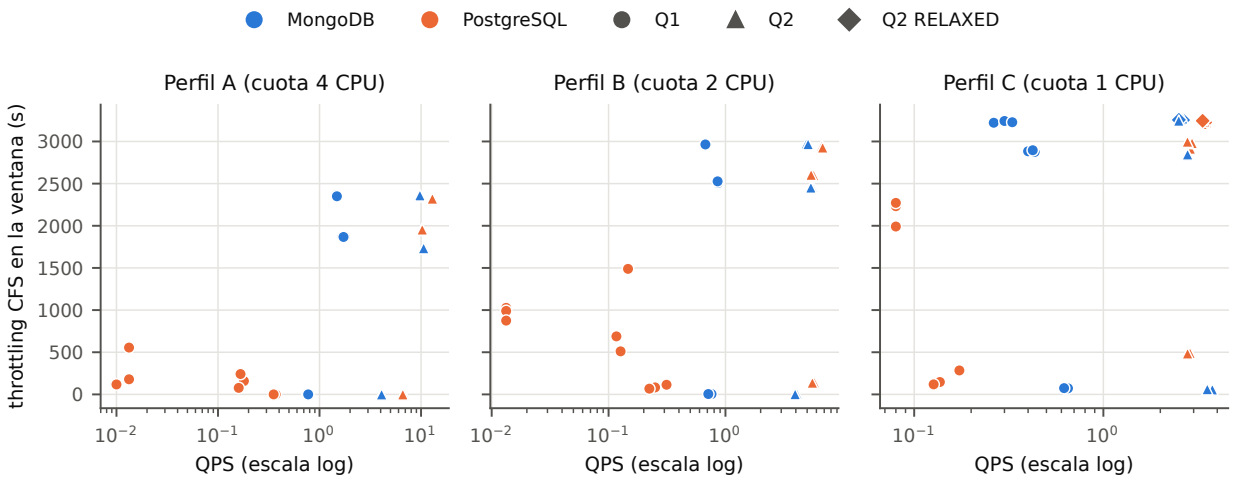


Figura 2: Throttling de CFS en la ventana (`throttled_usec` de `cpu.stat`) frente a QPS; una marca por corrida (114). Eje de QPS logarítmico.

grupo agota su cuota dentro de un periodo, esto indica que, con una sola consulta, MongoDB usa más de una CPU durante parte del tiempo.

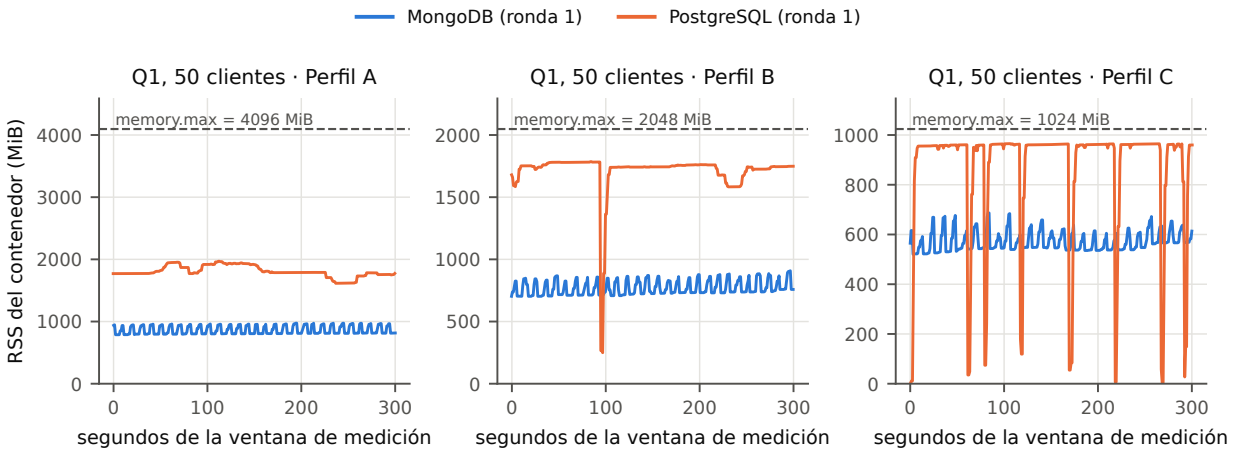


Figura 3: RSS del contenedor cada segundo (cAdvisor) durante la ventana de medición, Q1 con 50 clientes, ronda 1. La línea discontinua es `memory.max`. Las caídas de PostgreSQL en B y C son OOM kills: el kernel mata un proceso y PostgreSQL reinicia todos sus procesos.

Curva RSS (figura 3). MongoDB se mantiene por debajo de 1 GiB en los tres perfiles (entre unos 550 y 980 MiB), con oscilaciones regulares y sin acercarse al límite. Con 50 clientes en Q1, PostgreSQL ocupa entre 1 800 y 1 970 MiB de RSS en el perfil A: casi todo el límite de B (2 048 MiB) y casi el doble que el de C. En B la RSS sube hasta cerca de 1 780 MiB y cae casi a cero cuando ocurre el OOM kill; en C toca los 960 MiB y cae una y otra vez durante toda la ventana.

5. Planes de ejecución

Los planes se guardaron antes de medir, con el perfil A y un solo cliente, usando `runner/explain_plans.py`. MongoDB, con `explain` en modo `executionStats`; PostgreSQL con `EXPLAIN` y las opciones `ANALYZE`, `BUFFERS`, `VERBOSE` y `SETTINGS`. Están completos en `evidence/plans/`. La tabla 7 los pone lado a lado. Cada valor sale de esos archivos y corresponde a una sola ejecución con el perfil A y 1 cliente: *no* es lo medido en la matriz (secciones 3 y 4).

Tabla 7: Planes lado a lado (`evidence/plans`, perfil A, 1 cliente, una ejecución; no es lo medido en la matriz)

	Motor	Operador principal	Examinados	Ordenamiento o agrupamiento	Memoria o disco	Tiempo (ms)
Q1	MongoDB	`COLLSCAN` → `$unwind` → `$group`	500 000 docs.	`$group`, 90 grupos	memoria: pico 42 870 B; `usedDisk`: false	2 535
Q1	PostgreSQL	`Parallel Seq Scan` → `Sort` → `Incremental Sort` → `Aggregate (Sorted)`	500 000 filas	Sort: `external merge` (3 procesos); Incremental Sort: `external merge`	disco: Sort 186 640/199 600/198 264 kB; 201 090 bloques temporales escritos	3 838,8
Q2	MongoDB	`IXSCAN` (`fecha_1`) → `FETCH` → `SORT`	227 094 docs. (193 203 pasan el filtro)	`SORT` con límite 1 000	memoria: 1 598 598 B ordenados, pico 1 600 108 B (límite 104 857 600 B); `usedDisk`: false	266
Q2	PostgreSQL	`Parallel Index Scan` (`ventas_fecha_idx`) → `Sort` → `Gather Merge` → `Limit`	227 094 filas (193 203 pasan el filtro)	`top-N heapsort`	memoria: 3 071/3 040/3 059 kB; 0 bloques temporales	185,1

PostgreSQL: valores de `EXPLAIN` en JSON; filas examinadas = (filas + filas eliminadas por el filtro) × loops del nodo de lectura; espacio de ordenamiento del líder y de cada trabajador. MongoDB: `explain` en modo `executionStats`. El tiempo es el de esa única ejecución del plan. El extracto en texto de Q1 que sigue (`postgres_Q1.txt`) es una segunda ejecución, por eso sus tiempos y tamaños difieren un poco.

Q1 en PostgreSQL vuelca a disco. El planificador ordena los documentos JSONB completos (`width=1172`) por `tipo_pago` en un `Sort` paralelo, antes de desanidar los artículos, para alimentar un `Incremental Sort` y un `GroupAggregate`. Cada uno de los 3 procesos (líder y 2 trabajadores) ordena unos 190–200 MB, muy por encima de `work_mem` (4 MB por defecto):

```
GroupAggregate (actual time=1971.048..3137.176 rows=90.00 loops=1)
  Buffers: shared hit=358 read=80006, temp read=200883 written=201096
  -> Incremental Sort (... rows=674686.00 loops=1)
       Pre-sorted Groups: 3 Sort Method: external merge Average Disk: 141280kB
       -> Nested Loop
            -> Gather Merge (Workers Launched: 2)
                 -> Sort (... rows=166666.67 loops=3)
                      Sort Key: ((ventas.data ->> 'tipo_pago'::text))
                      Sort Method: external merge Disk: 190536kB
                      Worker 0: Sort Method: external merge Disk: 202128kB
                      Worker 1: Sort Method: external merge Disk: 191872kB
                      -> Parallel Seq Scan on public.ventas
            -> Function Scan on jsonb_array_elements item (loops=500000)
Execution Time: 3211.518 ms
```

La ejecución del `EXPLAIN` escribió 840 851 456 bytes temporales (`pg_stat_database`), unos 0,78 GiB. En la matriz, el volumen por consulta con 1 cliente es el mismo: 87,7 GiB para 110 consultas en el perfil A, 60,1 GiB para 75 en B y 33,1 GiB para 41 en C, es decir, unos 0,8 GiB por ejecución. Subir `work_mem` a 64 MB solo en la sesión no elimina el volcado: el plan sigue con `external merge` y escribe 804 732 928 bytes (`postgres_Q1_WM64.txt`; esta variante es solo evidencia y no se midió en la matriz).

Q1 en MongoDB no vuelca. El plan es `COLLSCAN` sobre los 500 000 documentos, `$unwind`, `$match`, `$project` y `$group` con 90 grupos; `usedDisk: false` y 2 535 ms en el perfil A. El `$group` solo guarda los 90 acumuladores, así que su memoria no depende del tamaño de la colección, y nunca reordena los documentos.

Q2 en ambos motores. Los dos usan el índice de `fecha` y un ordenamiento Top-N en memoria:

- PostgreSQL: `Parallel Index Scan using ventas_fecha_idx`, filtro de `es_fiado` y `Sort Method: top-N heapsort Memory: 3070kB` en cada proceso; 0 bytes temporales con 4 MB y con 64 MB.
- MongoDB: `IXSCAN` sobre `fecha`, 227 094 documentos examinados, `SORT` con 1 598 598 bytes ordenados (el límite por etapa es 104 857 600 bytes) y `usedDisk: false` con `allowDiskUse` en `false` y en `true`.

Como Q2 solo conserva las 1 000 mejores filas, el ordenamiento nunca llena la memoria disponible, y las variantes restrictiva y relajada dan el mismo plan. Eso explica que `Q2_RELAXED` no cambie las métricas (sección 3). El criterio para medir más celdas relajadas (fijado antes de medir: que el plan cambiara entre variantes) no se cumplió.

6. Interpretación: límites de memoria, volcado a disco y throttling

6.1. Respuestas a las preguntas del enunciado (M1, M2, M3)

M1. ¿Cómo cambian la latencia P95 y P99 de las agregaciones sobre documentos anidados al limitar CPU y memoria? En la agregación Q1, MongoDB aumenta su latencia casi en proporción a la cuota de CPU cuando hay concurrencia: con 10 clientes el P99 pasa de 6 090 a 11 808 y 24 799 ms (perfiles A, B y C) y el P95 de 5 923 a 11 505 y 23 801 ms; con 50 clientes el P99 pasa de 35 077 a 66 694 y 137 694 ms. Con 1 cliente el cambio es menor (P99 de 1 411, 1 429 y 2 063 ms). PostgreSQL parte de latencias más altas por el volcado a disco (P99 con 1 cliente: 2 790, 5 343 y 8 497 ms; P95: 2 744, 5 038 y 8 179 ms) y con 50 clientes deja de responder: en B termina en OOM con 4 consultas por ventana y en C en OOM sin ninguna. Con 10 clientes su P99 no sigue a la cuota (82 581, 74 662 y 105 002 ms), porque espera al disco. Detalle en la tabla 4 y la figura 1 (sección 4).

M2. ¿Qué motor usa de forma más eficiente los buffers de ordenamiento y agrupamiento antes de volcar a disco o fallar por OOM? En Q1, MongoDB: su `$group` alcanza un pico de 42 870 B para 90 grupos y no usa disco (tabla 7), y en sus 57 corridas no hubo volcado ni OOM (tabla 5). PostgreSQL ordena los documentos completos con un `work_mem` de 4 MB, vuelca unos 0,8 GiB por ejecución en los tres perfiles (21 corridas `spill`) y con 50 clientes falla por OOM en B y C (6 corridas). En Q2 no hay diferencia: los dos resuelven el Top-1000 en memoria (unos 3 MB en PostgreSQL y 1,6 MB en MongoDB) sin volcar, con ambas variantes de memoria (sección 5).

M3. ¿Cuál es la huella de CPU y memoria por cada 1 000 documentos procesados en cada motor? Con la definición adoptada en la sección 3 (el enunciado no la precisa), en Q1 con 1 cliente PostgreSQL usa 8,7–11,8 ms de CPU por cada 1 000 documentos y MongoDB 2,9–3,2 ms; en Q2 los dos quedan entre 1,1 y 1,8 ms (tabla 6). En memoria, MongoDB mantiene una RSS mediana de 493 a 974 MiB y PostgreSQL usa menos RSS en Q2 (mediana de 20 a 298 MiB), con la salvedad sobre la memoria compartida que se indica en la sección 3. Las celdas de PostgreSQL Q1 con 50 clientes no tienen una huella comparable (pocas consultas u OOM).

6.2. Análisis

El cuello de botella de PostgreSQL Q1 es el plan, no el límite de memoria. El volcado aparece desde el perfil A con 1 cliente (4 GiB de límite y 0,1 % de periodos con throttling) y con el mismo volumen, unos 0,8 GiB por ejecución, en los tres perfiles. Lo que lo provoca es la decisión del planificador de ordenar los documentos completos antes de desanidarlos, con un `work_mem` de 4 MB por nodo de ordenamiento; el límite del contenedor no interviene. Por eso su throttling es bajo y su QPS cae con la concurrencia: con 10 clientes en A el QPS baja de 0,37 a 0,17, aunque haya CPU libre (86 % de los periodos sin throttling). Las consultas se forman detrás del disco.

El OOM aparece cuando la memoria de 50 sesiones no cabe en el límite. En el perfil A, 50 clientes de Q1 llevan la RSS de PostgreSQL a 1 970 MiB y `memory.peak` llega a los 4 096 MiB del límite; no hay OOM porque esa diferencia es caché de página de los archivos temporales, que el kernel puede liberar. En B el límite es de 2 048 MiB: la RSS sube a unos 1 780 MiB y, sumada a la caché de los temporales, toca el límite; sin swap (`memory.swap.max = 0`), el kernel mata un proceso (5 OOM kills en 3 rondas). En C (1 024 MiB) la RSS llega a 960 MiB en segundos y hay 144 OOM kills en 3 rondas. En los registros de PostgreSQL se ve `client backend ...was terminated by signal 9: Killed` seguido de `all server processes terminated; reinitializing` y de una recuperación automática: cada OOM kill tira todas las sesiones, no solo la que lo provocó, y por eso hay errores de conexión (12 168 en C y 428 en B). *Hipótesis no verificada:* la RSS por sesión viene de los búferes de ordenamiento de hasta 3 procesos por consulta y de la compilación JIT que muestra el plan (191 ms de JIT por ejecución).

MongoDB se degrada por CPU, no por memoria. MongoDB no volcó a disco ni tuvo OOM; su RSS se mantuvo por debajo de 1 GiB en todos los perfiles. En Q1 su QPS con 10 clientes es casi proporcional a la cuota de CPU (1,72/0,87/0,42 para 4/2/1 CPU), y su P99 con 50 clientes crece en la misma proporción (35, 67 y 138 s). En el perfil C su `memory.peak` llega a 1 024 MiB con Q1 y 50 clientes sin OOM: la memoria que excede la RSS es caché que el kernel recupera.

El throttling mide saturación de la cuota, no trabajo perdido. Con 10 o más clientes, toda corrida limitada por CPU tiene throttling en el 100 % de los periodos, y los segundos acumulados crecen al bajar la cuota (por ejemplo, Q2 con 50 clientes en PostgreSQL: 2 322, 2 935 y 3 237 s en A, B y C). Como el contador suma el tiempo de espera de todas las CPU, sirve para comparar celdas y para ver cuándo la cuota está saturada, pero no se puede leer como "segundos de reloj perdidos". El indicador que traduce el throttling al usuario es el QPS, que en esas corridas escala con la cuota.

Q2: con un Top-1000 no hay presión de memoria. Ninguna variante vuelca a disco porque el ordenamiento Top-N solo conserva 1 000 filas (unos 3 MB en PostgreSQL y 1,6 MB en MongoDB). La diferencia entre motores en Q2 es de CPU: PostgreSQL usa trabajadores paralelos y responde antes con 1 cliente; con 50 clientes saca unas 1,3 veces más QPS con la misma cuota (1,33, 1,31 y 1,32 en A, B y C). *Hipótesis no verificada:* el costo por consulta de PostgreSQL en Q2 es menor porque lee los documentos JSONB desde el índice y descarta pronto los que no entran en el Top-N.

7. Recomendaciones técnicas

Para el sistema de abarrotes, a partir de lo medido:

1. **Agregaciones sobre los artículos de las ventas (tipo Q1).** Con el esquema JSONB tal como está, no conviene ejecutarlas en PostgreSQL de forma concurrente: cada ejecución escribe unos 0,8 GiB temporales y con 50 clientes provocó OOM con 1 y 2 GiB. Las opciones, en orden de esfuerzo:
   - Ejecutarlas en MongoDB, que las resolvió sin disco y sin OOM en todos los perfiles.
   - En PostgreSQL, guardar los artículos en una tabla relacional (o en una columna generada) para no ordenar el documento JSONB completo. *No se midió.*
   - Limitar la concurrencia de estos reportes (por ejemplo, una cola o un *pool* de 1 a 2 conexiones), porque el volumen temporal crece con los clientes.
2. **No confiar en subir `work_mem`.** Con 64 MB el plan de Q1 sigue volcando; además, `work_mem` se reserva por nodo y por proceso, y con muchas sesiones el riesgo de OOM aumenta. Cualquier cambio debe probarse con el `EXPLAIN` y con la concurrencia real.

3. **Dimensionar la memoria de PostgreSQL por sesión, no por consulta.** Con 50 sesiones de Q1, la RSS llegó a unos 2 GiB; un contenedor de 1–2 GiB no basta. Con límites estrictos, conviene un *pooler* que limite las conexiones activas.
4. **Consultas de listado tipo Q2.** Ambos motores las resuelven en memoria; si el objetivo es la latencia con pocos clientes, PostgreSQL fue más rápido. Para más clientes, lo que importa es la cuota de CPU: el QPS escaló casi en proporción a ella. Las variantes de memoria (`allowDiskUse`, `work_mem`) no aportan nada aquí.
5. **Vigilar el throttling junto con el QPS.** En producción, un 100 % de periodos con throttling indica que la cuota de CPU está saturada; si coincide con un QPS que no sube, hace falta más CPU. Un QPS bajo con poco throttling, como PostgreSQL Q1, apunta al disco o al plan.

Trabajo futuro (no medido): esquema relacional de los artículos en PostgreSQL, efecto de desactivar JIT o el paralelismo en Q1, índices sobre `total` para Q2, y repetir la matriz en Linux nativo.

Desviaciones y limitaciones

1. **Entorno.** Docker Desktop sobre WSL2 en Windows, no Linux nativo. La VM tiene 12 CPU y 7,39 GiB; el cliente de carga corre en Windows y comparte el procesador con la VM.
2. **Dataset adaptado al proyecto de abarrotes.** En lugar de las órdenes del enunciado: `es_fiado = false` reemplaza al estado `completed`; `tipo_pago` a país; el proveedor del artículo a categoría; `folio` desempata el orden de Q2; el `id` de PostgreSQL es un UUID determinista (uuid5) calculado del `_id`. Cada artículo lleva un subdocumento `producto` para llegar a 4 niveles. Datos sintéticos; 940 B por venta en JSON.
3. **Tolerancia en Q1.** Las sumas de MongoDB son `double` y las de PostgreSQL `numeric`; se aceptó una diferencia de hasta 1 centavo (la máxima fue $9{,}3 \times 10^{-10}$).
4. **`shm_size: 1g` en PostgreSQL.** El `/dev/shm` de 64 MB que Docker da por defecto corta las consultas paralelas. Es un ajuste de Docker, no del motor.
5. **Swap.** `mem_swappiness: 0` no se aplica en cgroups v2 (Docker lo ignora); el swap se anuló con `memswap_limit` igual a la memoria y se verificó `memory.swap.max = 0`.
6. **Puerto de MongoDB.** El contenedor se publica en el 27018 porque el 27017 lo usa un servicio MongoDB de Windows, que se mantuvo detenido durante la matriz.
7. **Throttling.** El valor del CSV sale de `cpu.stat` y suma todas las CPU, por lo que puede superar los 300 s de la ventana. La serie de cada segundo de cAdvisor tiene una resolución efectiva de unos 2 s.
8. **Percentiles con pocas consultas.** En PostgreSQL Q1 con 50 clientes terminan 3 o 4 consultas por ventana (perfiles A y B): su P95 y su P99 son el máximo. Varias celdas de Q1 tienen menos de 100 consultas (marcadas con †).
9. **Temporales tras un OOM.** PostgreSQL reinicia sus estadísticas al recuperarse de un OOM kill, así que en las corridas `oom` los bytes temporales no se pudieron medir: `disk_spill` queda vacío y nunca se estima.
10. **Serie de RSS de cada segundo.** El runner consultó cAdvisor por nombre de contenedor y tomó la primera serie devuelta; cuando el contenedor se recrea, Prometheus conserva unos minutos la serie del contenedor anterior con el mismo nombre. Una auditoría posterior (`analysis/auditar_rss.py`), que filtra por el id del contenedor de cada corrida, encontró que en 24 de las 114 corridas el `serie_1s.csv` guardado es de otro contenedor; la figura 3 usa las series corregidas (`report/generado/rss_corregida/`). El `max_rss_mb` del CSV es correcto en las 114 corridas, y el throttling del CSV sale del cgroup, así que no se ve afectado.

11. **Piloto.** En el CSV del piloto (`results/piloto/`, fuera de la matriz) la fila de PostgreSQL C Q1 con 50 clientes, que terminó en `oom`, quedó con `disk_spill = false` sin haberse medido. Se corrigió el runner antes de la matriz y el CSV del piloto se conserva tal como salió.
12. **Lazo cerrado.** Los clientes lanzan la siguiente consulta en cuanto termina la anterior, sin tiempo de espera; con 50 clientes la latencia refleja sobre todo la cola, no la velocidad de una consulta aislada.
13. **Pausa.** La matriz se pausó una vez entre corridas (11:59 a 12:49). No hubo corridas a medias ni descartes.
14. **`memory.peak`.** Es el pico del cgroup desde que arranca el contenedor, incluido el calentamiento, y cuenta caché de página; no equivale a la RSS.
15. **Huella por cada 1 000 documentos (M3).** Es una definición adoptada en este reporte, porque el enunciado no la precisa. El CPU de la ventana incluye el de las consultas en vuelo que se cancelan al cerrarla, lo que sesga la huella hacia arriba, sobre todo con pocas consultas ok, y no incluye el cliente, que corre en Windows. Los documentos por consulta salen de los planes con el perfil A y 1 cliente y se asumen constantes. La razón de RSS por documentos/s es una interpretación del enunciado.

A. Reproducción y archivos

Desde la raíz del repositorio, con el entorno de Python de `requirements.txt`:

```
docker compose --profile telemetry up -d # Prometheus, cAdvisor y Node Exporter
python synthetic_data_generator.py # 500 000 ventas, semilla 20261101
python runner/load_data.py # carga e índices en ambos motores
python runner/verificar_f2.py # verificación del baseline
python runner/explain_plans.py --out evidence/plans --queries Q1 Q1_WM64 Q2 Q2_RELAXED
./run_benchmarks.sh --rondas 3 --timeout 300 # matriz completa (orquestador)
python analysis/auditar_rss.py # auditoría de la RSS (con Prometheus)
python analysis/consolidar.py # tablas y figuras de este reporte
```

- `results/benchmark_results.csv`: las 114 filas (14 columnas fijas).
- `results/ronda_0n/runs/<run_id>/`: `meta.json`, `latencias.csv`, `serie_1s.csv` y, en `oom`, `docker_logs.txt`.
- `results/bitacora.log`: eventos con hora (inicio, pausa, reanudación, respaldos).
- `evidence/INDEX.md`: una línea por cada corrida `spill` u `oom` con sus archivos.
- `evidence/plans/`, `evidence/verificacion/`, `evidence/entorno/`.

Los manifiestos SHA-256 de `results/manifiestos/` verifican con `sha256sum -c` en un clon del repositorio, salvo cuatro archivos que el orquestador actualizó después de hacer el respaldo de la ronda 3: `bitacora.log`, `estado.txt`, `resumen.md` y `orquestador_matriz.out`.